PYTHON FOR (OPEN) NEUROSCIENCE

LECTURE P.3 — *PROMPT FORMULATION FOR SCIENTIFIC CODING*

MODULE P - PROJECT

Sara Assecondi
Department of Psychology and Cognitive Sciences

> ***Goal of this lecture:*** Treat prompts as experimental specifications, not casual requests.

Why prompt formulation matters

LLM = Large Language **Model**

Common failure

“Analyze my EEG data” is underspecified: task, assumptions, preprocessing, and outputs remain ambiguous.

Systems view

A prompt is an executable specification delivered to a stochastic model.

Operational goal

Increase reliability, controllability, and auditability of model outputs.

- Use prompts to define task boundaries, context, and admissible outputs.
- Optimize coding prompts for correctness, inspectability, and reproducibility.
- Treat prompt design as interface design between scientific intent and model behavior.

A prompt is a specification

Task
Define the exact action

→

Context
Provide domain facts and data layout

→

Constraints
State rules, exclusions, priorities

→

Output schema
Specify structure, fields, style

→

Verification
Ask for checks, tests, uncertainty signals

Weak prompt
Analyze this neural dataset.

Stronger spec
Task: compute per-neuron firing rate by condition
Context: one CSV, columns = neuron_id, t_ms, condition
Constraints: pandas + numpy only, no plotting
Output: one function, one test, brief assumptions

Block design for a prompt system

Use the same **modular thinking** as a neuroscience pipeline.

1. Problem formulation

Scientific objective, unit of analysis, acceptable abstraction

→

2. Context assembly

Data schema, codebase constraints, domain assumptions, prior decisions

→

3. Instruction layer

Exact task, ordered priorities, negative constraints, output contract

→

4. Model + tools

Model call, retrieval or execution tools, bounded permissions

→

5. Validation layer

Syntax, tests, format compliance, scientific plausibility

Anatomy of a high-quality technical prompt

Every segment should reduce ambiguity

Context has highest leverage

Schema, file names, variables, library limits, and interfaces improve correctness more than stylistic phrasing.

Constraints expose priorities

Tell the model what not to do, how to handle uncertainty, and what to optimize.

Structured outputs are testable

If you can parse it, you can evaluate it: ask for lists, schemas, functions, or stepwise deliverables.

Template

Role: coding assistant for neuroscience analysis

Objective
- implement a specific task

Context
- data layout
- experimental design
- available libraries

Constraints
- do not fabricate files or results
- preserve variable names where possible
- state assumptions explicitly

Output
1. short plan
2. code
3. minimal test
4. caveats

Prompting patterns for neuroscience coding tasks

Different task classes need different prompt architectures.

Analysis planning

Ask for decomposition: inputs, transformations, outputs, tests, and likely failure points before code is written.

Code generation

Pin down libraries, data schema, function signatures, and edge cases. Request tests or assertions.

Debugging

Provide traceback, minimal reproducer, expected behavior, and what changed. Ask for ranked hypotheses, then a fix.

Interpretation / writing

Provide result objects and guard against overclaiming. Require uncertainty language and separation of result vs inference.

Example: debugging prompt

Task: diagnose a shape mismatch in my MNE pipeline
Context: epochs.get_data() returns (n_trials, n_channels, n_times)
Error: ValueError: operands could not be broadcast together
What changed: baseline correction was added after averaging
Output:
- 3 ranked hypotheses
- exact code fix
- one assert to catch this earlier

Iterative prompt refinement

Prompt engineering is empirical: design → run → inspect → revise.

Baseline prompt → **Inspect errors** → **Classify failure mode** → **Add missing constraint** → **Re-run and compare**

Typical failure modes
Missing context, underspecified output format, weak negative constraints on what to avoid, absent tests, or excessive task scope.

Good practice
Revise one variable at a time when possible and compare outputs on a small representative task set.

If the output is fluent but wrong, improve the specification before escalating model complexity.

Evaluation: how do we know a prompt is good?

For code tasks

Compiles or runs
Passes minimal tests
Uses allowed libraries
Matches requested API
Handles stated edge cases

For scientific writing

Factual fidelity
No overclaiming
Explicit uncertainty
Correct separation of method, result, inference
Citations when required

For structured outputs

Schema validity
Field completeness
No hallucinated fields
Deterministic formatting
Easy downstream parsing

Failure modes and risks in research settings

LLMs are helpful accelerators, but they are not ground truth.

Hallucinated APIs or file paths
Model invents functions, packages, columns, or filenames that do not exist.

Scientific overreach
Interpretation exceeds what the statistics support → cautious language and human review of conclusions.

Silent assumption injection
The model fills gaps without clearly flagging assumptions → require explicit assumptions

Data leakage / privacy
Sensitive data pasted into prompts may violate lab or patient-data rules.

Brittle dependence on phrasing
A solution works only for one wording and fails under small changes.

Exercise 1: Design a prompt system, not just a single prompt.

Scenario

You have calcium imaging data from multiple animals across two stimulus conditions. You want help generating a Python analysis scaffold that loads data, computes ΔF/F summaries, compares conditions, and produces a figure.

task A · Problem formulation

- scientific objective
- unit of analysis
- desired deliverable
- what must not be assumed

task B · Context block

- file structure
- variable names
- available libraries
- expected outputs

task C · Validation block

- one test
- one schema check
- one failure condition
- one required caveat

? explain why each block changes model behavior and reduces a specific failure mode.

Exercise 2: Design a prompt system, not just a single prompt.

Scenario

You have spike train data from multiple neurons recorded during two behavioral conditions: **rest** and **movement**. You want help generating a Python analysis scaffold that:

- loads the data
- computes firing-rate summaries
- compares the two conditions
- produces one figure

Your goal is to design a prompt system that would let a coding model generate a reliable first draft.